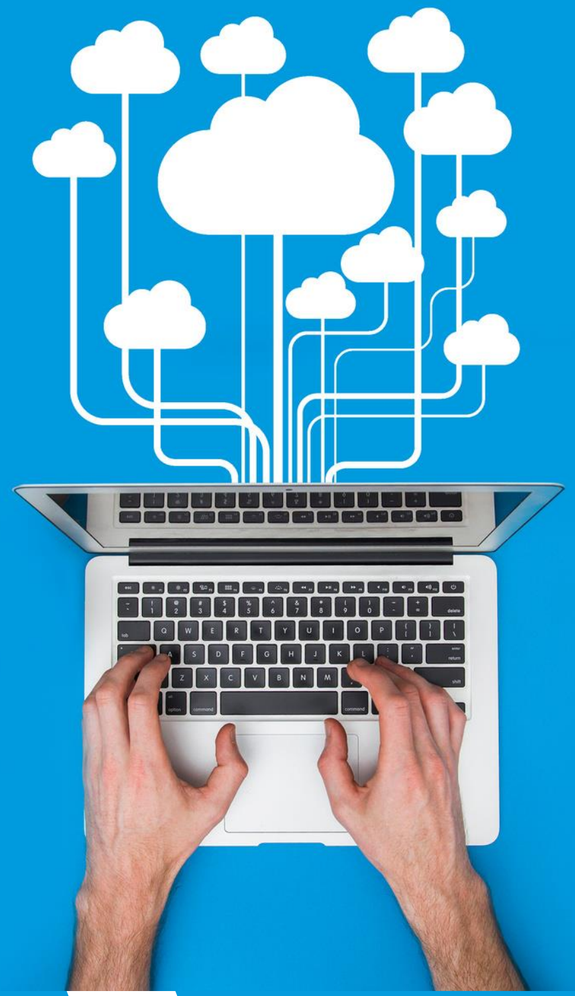
PYTHON PROGRAMMING



21 Days Challenge

Day-15

Day-15

PYTHON OOP : POLYMORPHISM & ABSTRACTION



Instructor



Abdullah Al Zubaer
Artificial Intelligence & Data Science
Gobeshona Learning Academy.
B.Sc. in Internet of Things and Robotics Engineering
Gazipur Digital University, Gazipur, Bangladesh

Table Of Content

- Polymorphism
- Method Overloading
- Method Overriding
- Operator Overloading
- Duck Typing
- Abstract

Polymorphism

Polymorphism means using one name for many forms — the same method or function can do different things depending on the object.

```
len("Hello")

len([1, 2, 3])
```



Why do we need Polymorphism?

- **Consistent interface** → same method name (e.g., sound() or move())
- **Dynamic behavior** → each object responds in its own way.
- **Loose coupling** → we don't care about object type, just the method.
- **Reusability & testing** → code is flexible, extendable, and easy to maintain.

Types of Polymorphism

- Compile-time Polymorphism (Overloading)
- Runtime Polymorphism (Overriding)
- Built-in Functions Polymorphism
- Functions (Duck Typing)
- Operator Polymorphism (Overloading Operators)

Method Overloading

Writing multiple methods with the same name but different numbers of arguments.

```
class Test:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

```
t = Test()

print(t.add(1, 2, 3)) # works

print(t.add(1, 2)) # Error
```

Method Overloading

```python
class Calculator:
    def add(self, *args):
        return sum(args)
```

```python
calc = Calculator()
print(calc.add())           # 0
print(calc.add(5))          # 5
print(calc.add(5, 10))      # 15
print(calc.add(5, 10, 20)) #35
```

Method Overriding

When a child class provides its own version of a method that already exists in the parent class. The method name and parameters are the same, but the behavior is different.

```
class Animal:
    def sound(self):
        return "Some generic sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"

animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
```

Operator Overloading

```
print(5 + 10)
print("Hello " + "World")
print([1, 2] + [3, 4])
```

OutPut:

```
15
Hello World
[1, 2, 3, 4]
```

Duck Typing

A style of programming where an object’s suitability is determined by whether it has the needed methods or properties, not by its type.

```
class Pen:
    def use(self):
        return "Writing"

class Eraser:
    def use(self):
        return "Erasing"
```

```
def perform_task(tool):
    print(tool.use())

perform_task(Pen())
perform_task(Eraser())
```

Abstract

An abstract class is a blueprint for other classes that cannot be instantiated on its own and serves as a base class for inheritance.

- Abstract classes **cannot create objects directly**.
- They can have **abstract methods** and **regular methods**.
- Abstract methods **must be implemented** in any subclass. otherwise, the subclass cannot be instantiated.
- Python provides the abc module to define abstract classes.

Abstract

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):  # abstract method
        pass

def breathe(self):  # Concrete method
        return "Breathing..."
```



ASSIGNMENT

- Use a single function add_any() with default arguments to show compile-time polymorphism.
- Create a base class Shape and override area() in Circle and Rectangle to show runtime polymorphism.
- Use built-in functions like len() and sorted() on different data types.
- Write a function run_engine() that calls .start() on both CarEngine and BusEngine objects to show duck typing.
- Create a class Vec2 that overloads + and * operators to add vectors and multiply by a scalar.

QUESTION SESSION

THANK YOU

https://gobeshona.com.bd/